ESCOLA POLIEDRO · DOCUMENTO TÉCNICO

Poliedraw

Plataforma educacional para geração de imagens com Google Gemini 2.5 Flash Image.

Versão
1.0.0

Data
21 de outubro de 2026

Stack
Node.js · Express 5 · MySQL · HTML/CSS/JS

Entrega
Arquitetura, API, dados e operação

Sumário executivo

Uma visão rápida do produto, arquitetura e contratos técnicos.

O Poliedraw transforma descrições em linguagem natural em imagens educacionais. Professores e alunos possuem fluxos separados, podem gerar imagens com Gemini e salvar criações em uma galeria.

1. Visão geral e propósito 03

2. Arquitetura técnica 04

3. API e endpoints 06

4. Modelo de dados 09

5. Configuração e execução 11

6. Segurança e evolução 13

Escopo funcional

Geração

Prompt textual, resposta multimodal e download local.

Perfis

Professor e aluno com entradas de login distintas.

Galeria

Persistência, consulta e exclusão de imagens.

Gestão

CRUD de alunos no fluxo administrativo.

Precisão: SQLite está instalado, mas o runtime atual usa somente MySQL.

1. Visão geral e propósito

O Poliedraw aproxima criatividade, tecnologia e aprendizagem em uma ferramenta escolar simples. A interface permite descrever uma ideia, visualizar a imagem gerada, baixá-la e manter uma galeria vinculada ao perfil.

Requisitos

ID	Entrega
RF01–RF02	Login independente para professor e aluno.
RF03–RF05	Redirecionamento por perfil, envio de prompt e exibição Gemini.
RF06–RF07	Persistência e consulta das imagens.
RF08–RF09	Administração de alunos e exclusão de imagens.

Tecnologias

Camada	Implementação
IA	@google/genai com gemini-2.5-flash-image.
Banco	MySQL com mysql2/promise.
SQLite	Dependências instaladas; adaptador ainda não conectado.
UI	HTML5, CSS3 e JS em HTML/, CSS/ e JS/.

Requisitos não funcionais

- ES Modules e Express 5.
- JSON de até 50 MB para imagens Base64.
- Infraestrutura configurada por .env.
- CORS habilitado para integração local.

2. Arquitetura técnica



Fluxo de geração

1. Login em /professores ou /alunosLogin.
2. Frontend grava usuarioId, usuarioTipo e usuarioNome no localStorage.
3. frontGemini.js envia prompt para /api/gemini.
4. Backend aplica prompt de sistema e chama o Gemini.
5. Texto e inlineData voltam como texto e imagens.
6. Imagem é exibida, baixada e enviada a /salvarImagem.
7. Galeria usa /minhasImagens e /imagens/:id.

Organização

Arquivo	Responsabilidade
backend/conexao.js	Express, pool MySQL, tabelas e endpoints.
backend/gemini.js	Prompt do sistema e resposta multimodal.
BD/banco.sql	Schema e dados iniciais.
HTML/ CSS/ JS/	Páginas, estilos e comportamento.

> **Segurança atual:** não há sessão server-side, JWT, hash de senha ou middleware de autorização.

3. API · autenticação e imagens

Base local: http://localhost:3000. Entradas e respostas são JSON; imagens trafegam como Data URL/Base64.

POST /professores

Autentica professor. Sucesso: { success, id, nome }; credencial inválida: 200 { success:false }; banco: 500.

POST /alunosLogin

Mesmo contrato, consultando a tabela de alunos.

POST /salvarImagem

```
{ "id_usuario": 1, "tipo_usuario": "aluno", "prompt": "Mapa do ciclo da água", "imagem": "..." }
```

id_usuario e imagem são obrigatórios; ausência retorna 400. Sucesso retorna confirmação.

GET /minhasImagens?id=&tipo=

Exige os dois parâmetros, filtra por usuário/perfil e ordena por data_criacao DESC.

DELETE /imagens/:id

Exclui por id_imagem; registro ausente retorna 404.

> **Manutenção:** a rota DELETE /imagens/:id está declarada duas vezes com implementação equivalente.

3. API · administração e Gemini

GET /alunos

Retorna todos os registros de alunos.

POST /cadastrarAlunos

```
{ "nome": "Ana", "email": "ana@sistemapoliedro", "senha": "segredo" }
```

Retorna { success, id }; erros de inserção são 500 no código atual.

PUT /alunos/:id

Atualiza nome e e-mail. O e-mail precisa terminar em @sistemapoliedro; e-mail inválido: 400; ID ausente: 404.

DELETE /alunos/:id

Exclui por ID. Histórico referenciado: 400; ID inexistente: 404.

POST /api/gemini

```
{ "prompt": "Uma célula vegetal com suas estruturas identificadas" }
```

Sem prompt: 400. Sucesso: { texto, imagens: ["..."] }. Falha do Gemini: 500.

Regras didáticas

- Fidelidade, composição limpa e clareza visual.
- Neutralidade de estilo quando não solicitada.
- Interpretação educacional para pedidos vagos.
- Sem violência, sexualidade, conteúdo ofensivo, marcas reais, celebridades ou textos longos não pedidos.

4. Modelo de dados

BD/banco.sql é a referência declarativa; o backend também garante as tabelas no boot.

professores e alunos

Tabela	Campos
professores	id_professor INT PK auto; nome VARCHAR(100); email VARCHAR(100) UNIQUE; senha VARCHAR(255).
alunos	id_aluno INT PK auto; nome VARCHAR(100); email VARCHAR(100) UNIQUE; senha VARCHAR(255).

historico_pesquisa

Campo	Tipo / uso
id_pesquisa	INT PK auto incremento.
tipo_usuario	ENUM professor/aluno; perfil lógico.
id_usuario	INT; autor lógico.
termo_pesquisado	TEXT; prompt/termo.
data_pesquisa	DATETIME; data/hora.

Não há chave estrangeira; os vínculos são lógicos por tipo e ID.

imagens_geradas

Campo	Tipo / uso
id_imagem	INT PK auto incremento.
id_usuario / tipo_usuario	Proprietário lógico e perfil.
prompt	TEXT; entrada da geração.
imagem	LONGTEXT; Data URL/Base64.
data_criacao	DATETIME; ordenação.

5. Configuração e execução

Instalação

```
git clone https://github.com/MarcosGitCode/Poliedraw.git
cd Poliedraw
npm install
```

Arquivo .env

```
GEMINI_API_KEY=SUA_CHAVE_DE_API_DO_GEMINI
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASS=sua_senha
DB_NAME=poliedro_ai
PORT=3000
```

Não versione este arquivo nem publique segredos.

Banco

```
mysql -u root -p < BD/banco.sql
```

O script cria poliedro_ai, as quatro tabelas e dados de exemplo. O backend repete a garantia via CREATE TABLE IF NOT EXISTS.

Scripts npm

Comando	Uso
npm start	node backend/conexao.js.
npm run dev	nodemon backend/conexao.js; requer nodemon disponível.

O package.json observado não declara nodemon em devDependencies.

6. Segurança e evolução

Prioridade	Ação	Motivo
Alta	Hash de senhas e sessão/token.	Evitar exposição de credenciais e identidade.
Alta	Middleware por perfil e propriedade.	Impedir acesso cruzado a dados.
Alta	Rate limiting e limites de Base64.	Reduzir abuso, custo e memória.
Média	Validação de payload e MIME.	Contratos previsíveis.
Média	Testes automatizados.	Preservar endpoints.
Baixa	Adaptador SQLite real ou remoção da dependência.	Alinhar promessa ao runtime.

Fonte da evidência

Conteúdo produzido a partir de backend/, BD/banco.sql, HTML/, CSS/, JS/, README.md e package.json. A referência textual completa está em docs/DOCUMENTACAO_TECNICA.md; esta página é sua apresentação executiva para impressão.

> **Entrega:** DOCUMENTACAO_TECNICA.md, documentacao_poliedraw.html e documentacao_poliedraw.pdf.